SISTEMAS OPERATIVOS

Aula Teórica 8

Daniel Castro
2025/2026

UAlg
UNIVERSIDADE DO ALGARVE

Introduction to Scheduling (1)

In a multiprogrammed computer, frequently there are multiple processes or threads competing for CPU at the same time. This situation occurs whenever two or more of them are simultaneously in the ready state.

The part of the operating system that makes the choice of which process to runs next is called the **schedular**. The algorithm is called the **scheduling algorithm**.

Old days of batch systems, a batch job could be a request to run multiple programs in succession. Some mainframes still combines batch and timesharing service, requiring the schedular to decide whether a batch job or an interactive user atr a terminal should run next. CPU time was a scarce resource, so a good schedular can make the difference in the perceived performance and user satisfaction.

The personal computer introduces the following changes:
- Most of the time there is only one active process.
- Much faster computers over the years made the CPU a rarely scarce resource.
  As a consequence, scheduling does not matter much in a simple PC, the high processing rate of the CPU, easily running several programs at the same time , hardly matters which goes first to the user.

The situation changed with network servers. Multiple processes oftem do compete for the CPU, so scheduling matters again.

In Mobile Phones, CPU have become more powerfull, the memory more plentiful, but battery lifetime was not. So some schedulers try to optimize the power consumption.

Introduction to Scheduling (2)

In addition to picking the right process to run, the schedular also has to worry about making efficient the use of the CPU because process switching is expensive. Doing too many process switching per second can chew up a substancial amount of CPU time.

Some processes (called **CPU-bound**) spend most of their time computing, while others process (called **I/O bound**) spend most of their time waiting for I/O.

It is worth noting that if CPUs get faster, processes tend to get more I/O-bound. This effect occurs because CPUs are improving faster than hard disks.

Scheduling are needed:

1-When a new process is created, decision to run firth the parent->child;
2-When a process exist, some other process must be chosen to run, from the set of ready processes;
3-When a process blocks on I/O, semaphore, or some other reason;
4- When a I/O interrupt occurs. Hardware clock provides periodic interrupts at 50 or 60Hz (or higher frequency).

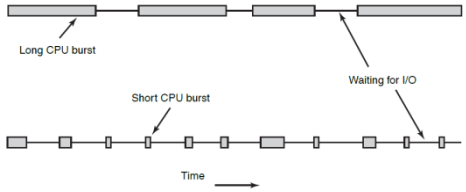


Figure 2-40. Bursts of CPU usage alternate with periods of waiting for I/O. (a) A CPU-bound process. (b) An I/O-bound process.

Scheduling algorithms can be divided into two categories with respect to how they deal with clock interrups:

Nonpreemptive – algorithm picks a process to run until it blocks;

Preemptive – algorithm pick a process and lets it run for a maximum of some fixed time.

Introduction to Scheduling (3)

In different environments different scheduling algorithms are needed, for different kinds of operating systems, different applications, and different goals.

Three environments worth distinguishing:

-Batch systems;
-Interactive users;
-Real-time constraints.

Fairness - comparable processes should get comparable service;

Throughput - n. of jobs per hour that the system completes;

Turnaround time - statistically average time from the moment that a batch job is submitted until the moment it is completed. It measures how long the average user has to wait for the output.

Response time - time between issuing a command and getting the result.

Proportionality- User have na inherent (but often incorrect) idea of how long things should take.

All systems
- Fairness—giving each process a fair share of the CPU
- Policy enforcement—seeing that stated policy is carried out
- Balance—keeping all parts of the system busy

Batch systems
- Throughput—maximize jobs per hour
- Turnaround time—minimize time between submission and termination
- CPU utilization—keep the CPU busy all the time

Interactive systems
- Response time—respond to requests quickly
- Proportionality—meet users' expectations

Real-time systems
- Meeting deadlines—avoid losing data
- Predictability—avoid quality degradation in multimedia systems

Figure 2-41. Some goals of the scheduling algorithm under different circumstances.

PREEMPTION

Refers to the action of taking the processor away from a running process due to the existence of another more priority process, or the opposite, if the scheduling is **non-preemptive**, the running process will only be switched if it finishes its *quantum* or if it gets blocked.

The purpose of preemption is precisely to allow a higher-priority process to respond more quickly to an event.

Preemption obviously has costs, and context switches contribute to performance degradation because they consume processor time. Some systems limit the use of preemption by not allowing a process to be removed from execution before a certain minimum execution time has elapsed.

In real-time systems, preemption is essential to ensure that a high-priority process executes immediately when it is unblocked. By acting consistently with the priorities of processes, it is possible to associate the most critical events with processes that, with high probability, will execute when the event is signaled to them through synchronization mechanisms, unless another higher-priority process is currently running, in which case it will have to wait until it becomes the highest priority.

Preemption is essential in real-time systems and is usually implemented in typical multi-user systems to provide greater dynamism in process management.

Scheduling in Batch Systems (1)

First-Come, First-Served
Probably the simplest os all scheduling algoritms.
Basically, there is a single queue of ready processes.
As any new job come in, it will go to the end of the queue.

Shortest Job First
Nonpreemptive batch algorithm that assumes the run times are known in advance.
Example:

a) By runnimg them in that order , the turnaround time for A is 8 min, for B is 12 min, for C is 16 min, and for D is 20 min, for an average of 14 minutes.
b) Considering the algorithm, the turnaround times are 4, 8, 12, 20 minutes for an average of 11 minutes.



Figure 2-42. An example of shortest-job-first scheduling. (a) Running four jobs in the original order. (b) Running them in shortest-job-first order.

Shortest Remaining Time Next
Preemptive version of Shortest Job First.
The schedular always choose the process whose remainig run time is the shortest.

Scheduling in Interactive Systems (1)

Algorithms for interactive systems like: personal computers, servers, etc.

Round-Robin Scheduling

One of the oldest, simplest, fairest and most widely used algorithms is round-robin. Each process is assigned a time interval, called its ***quantum*** *(time-slice)*, during which it is allowed to run.

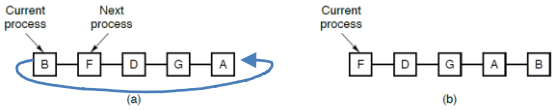


Figure 2-43. Round-robin scheduling. (a) The list of runnable processes. (b) The list of runnable processes after *B* uses up its quantum.

If the process is still running at the end of the *quantum*, the CPU is preemptive and given to another process.

If the process has blocked or finished before the *quantum* has elapsed, the CPU switching is done when the process blocks, of course.

All the schedular needs to do, is maintain a list of runnable processes Fig. 2-43(a). When the process uses up its *quantum*, it is put on the end of the list Fig. 2-43(b).

The only really interesting issue with round-robin is the length of the **quantum**.

Switching from one process to another requires a certain amount of time for doing administration – saving and loading registers and memory maps, updating various tables and lists, flushing and reloading the memory cache, and so on.

Scheduling in Interactive Systems (2)

Quantum or time-slice

This solution was first introduced in the CTSS (Compatible Time Sharing System) at MIT (1962); the management algorithm assigned each process a processor time *quantum*, managing the list of executable processes in a circular (round-robin) manner. This solution of extreme simplicity has the evident problem of easily leading to increasing response times in high-load situations, given the growing size of the list. Ran on the IBM 7094, which could hold only one process in memory each time.

Variable *quantum*

In many processes the quantum is fixed, leading to a simplification of implementation.
In other systems the duration of the *quantum* varies dynamically, calculated according to the priority of the process.

This idea had already appeared in relatively old systems such as TOP10 of the DEC-20 (1977), where the multi-queue system with three queues had an increasing *quantum* that varied inversely with the associated priority, and it reappeared in version 2.6.23 of Linux, where the quantum varies with priority.

Scheduling in Interactive Systems (3)

Typically in UNIX, the value of the *quantum* is about 100 msec.

Quantum was chosen as a compromise value:

-On one hand, too short value causes too many processes switches and lowers the CPU efficiency. It must be sufficiently high to minimize the relative weight of the latency associated with inter-process computation.

-On the other hand, for higher values, the effect of pseudo-parallelism could be lost, that is, the time multiplexing of the various processes would become noticeable. Also poor response to short interactive requests.

Quantum in Linux is a dynamic value but is also around 100msec.

In Windows Vista and Windows 7 it is 31msec, or 3x 31msec for all tasks of a process that becomes the foreground window.

A quantum around 20-50 msec is often a reasonable compromise.

QUESTÕES?

Daniel Castro
dcastro@ualg.pt

